

R for Clinical Programming: Comparison, Files, Functions, Git & Packages

A Detailed, Beginner-Friendly Walkthrough of `package_dev.R`

Ram

July 03, 2026

Contents

Introduction	2
Comparing Two Data Frames	2
Set up two sample data frames	2
The quickest check: <code>identical()</code>	3
Detailed differences with <code>diffdf</code>	3
An alternative view with <code>arsenal</code>	4
Talking to the User: <code>message</code>, <code>print</code>, <code>warning</code>, <code>stop</code>	4
File Handling Functions	5
Listing files: <code>dir()</code> and <code>list.files()</code>	5
Creating, checking, and deleting directories	5
Reading many SAS files at once	5
Inspecting and copying files	6
Writing Your Own Functions	7
The basic syntax	7
Documenting a function (<code>roxygen2</code>)	7
Three kinds of functions	8
Exploring Existing Packages and Functions	8
What is CRAN?	8
Discovering a package's contents	8
Looking inside a specific function	8

Git Integration with RStudio	9
The everyday cycle (in RStudio’s Git pane)	9
One-time configuration	9
Wiring a project to Git and GitHub from R	9
Building Your First Package	9
Install the development toolkit	10
Package-level documentation	10
The development loop (recap)	10
rlang: Tidy Evaluation for Beginners	10
Why it exists	11
Core rlang functions	11
Writing a function that takes a column name	11
The rlang “is_” predicate family	12
Putting It All Together	13
Summary	13
Further reading	14

Introduction

This tutorial explains, line by line and in plain language, the ideas in the `package_dev.R` script. It is written for **beginners** — every function is introduced with *what it does*, *why you’d use it*, and *a runnable example*.

The examples use a clinical-programming flavour (comparing a “development” data set against a “QC” data set, reading SDTM `.sas7bdat` files), but the R concepts apply to any field.

How to read this document. Chunks that create files, read from your disk, or talk to Git are marked `eval=FALSE` so the document knits safely. Copy them into your own RStudio session to run them for real. The small illustrative chunks evaluate live when you knit.

Comparing Two Data Frames

A daily task in clinical programming is **double programming**: two people build the same data set independently, and you check that the results match. R has several tools for comparing data frames.

Set up two sample data frames

```

# "development" version
adsl_dev <- data.frame(
  USUBJID = c("01", "02", "03", "04"),
  AGE     = c(34, 45, 29, 50)
)

# "QC" version - note subject 04 has AGE 51 instead of 50
adsl_qc <- data.frame(
  USUBJID = c("01", "02", "03", "04"),
  AGE     = c(34, 45, 29, 51)
)

adsl_dev
#>   USUBJID AGE
#> 1      01  34
#> 2      02  45
#> 3      03  29
#> 4      04  50
adsl_qc
#>   USUBJID AGE
#> 1      01  34
#> 2      02  45
#> 3      03  29
#> 4      04  51

```

The two frames are identical except for one value. Our goal is to have R *tell us* exactly where they differ.

The quickest check: `identical()`

`identical()` is base R. It returns a single TRUE/FALSE — the frames either match exactly or they don't. It won't tell you *where* they differ, but it's a fast first gate.

```

identical(adsl_dev, adsl_qc)  # FALSE, because AGE differs for subject 04
#> [1] FALSE

```

Detailed differences with `diffdf`

The **diffdf** package is built for exactly this job. `diffdf()` reports which variables differ, in which rows, and shows the mismatched values side by side.

```

install.packages("diffdf")  # one-time install
library(diffdf)

diffdf(adsl_dev, adsl_qc)

```

Typical output points you straight to the problem:

```

Differences found between the two datasets...
Variable No of Differences First Difference
AGE      1                  Row 4

```

```
...
VARIABLE: AGE
  ..ROW..  BASE  COMPARE
    4      50    51
```

You can **save the result** to an object to inspect or log it later:

```
diff_output <- diffdif(adsl_dev, adsl_qc)
diff_output  # re-print the stored comparison anytime
```

An alternative view with arsenal

The **arsenal** package's `comparedf()` gives a different, summary-style report. The `by =` argument tells it which column identifies a row (here the subject ID), so it compares like-for-like subjects rather than relying on row order.

```
install.packages("arsenal")  # one-time install
library(arsenal)

summary(comparedf(adsl_dev, adsl_qc, by = "USUBJID"))
```

`diffdif` vs `arsenal`: both are excellent. `diffdif` is compact and popular in pharma QC; `arsenal::comparedf()` produces a richer, publication-style summary. Use whichever your team standardises on.

Talking to the User: message, print, warning, stop

R has four related ways for code to communicate. Understanding the difference is important because they behave very differently — especially `stop()`.

Function	Purpose	Stops execution?	Goes to
<code>print()</code>	Show a value/result	No	Console (stdout)
<code>message()</code>	Friendly status/progress note	No	Console (stderr)
<code>warning()</code>	“Something looks off, but I continued”	No	Collected, shown after
<code>stop()</code>	Fatal error — halt immediately	Yes	Error stream

```
message("Comparison of adsl_dev and adsl_qc completed. Differences identified.")
print("Please review the output for details on the discrepancies.")
#> [1] "Please review the output for details on the discrepancies."
warning("Ensure the data frames are correctly aligned and differences are expected.")
stop("Please investigate the discrepancies before proceeding.")
#> Error:
#> ! Please investigate the discrepancies before proceeding.
```

A few beginner notes:

- `message()` is the polite way to print progress. Because it uses *stderr*, it won't get mixed into results you capture, and users can silence it with `suppressMessages()`.
- `warning()` does **not** stop your code. R holds warnings and shows them at the end; `warnings()` re-displays the most recent ones.
- `stop()` **halts execution immediately** — everything after it is skipped. Use it for conditions that make it unsafe to continue.

```
warnings()    # show all recent warnings generated in this session
```

File Handling Functions

Clinical work involves lots of files (data sets, outputs, logs). R can inspect, create, and manage them without leaving the console.

Listing files: `dir()` and `list.files()`

`dir()` and `list.files()` are the same function under two names. They list the contents of a directory.

```
dir()                # everything in the working directory
dir(path = ".", pattern = "\\..R$", full.names = FALSE) # only .R files, names only
list.files()         # identical to dir()
```

Two arguments you'll use constantly:

- `pattern` — a **regular expression** filter. `"\\..R$"` means “ends with `.R`”. The `\\.` is an escaped dot (a literal period), and `$` anchors it to the end.
- `full.names` — `TRUE` returns full paths (good for feeding into read functions); `FALSE` returns bare file names (good for display).

Creating, checking, and deleting directories

```
dir.create("comparison_results")    # make a new folder
dir.exists("comparison_results")    # TRUE if it now exists
unlink("comparison_results", recursive = TRUE) # delete folder AND its contents
```

`recursive = TRUE` is required to remove a folder that contains files. **Be careful** — `unlink()` deletes permanently, with no recycle bin.

Reading many SAS files at once

This is the most powerful pattern in the script. The goal: read *every* `.sas7bdat` file in a folder into a single named list.

Step 1 — get the full paths of all SAS files:

```

sas_paths <- list.files(
  path      = "~/R_info/sas2R_s/data/sdtm",
  pattern   = "\\\\.sas7bdat$",      # only SAS transport files
  full.names = TRUE                # full paths, so read_sas can find them
)
sas_paths

```

Step 2 — read them all. `lapply()` applies a function to every element of a vector and returns a list. Here it reads each path with `haven::read_sas()`:

```

library(haven)

# Modern pipe style (>): take the paths, then read each one
sdtm_lst <- list.files("~/R_info/sas2R_s/data/sdtm",
  pattern = "\\\\.sas7bdat$", full.names = TRUE) |>
  lapply(haven::read_sas)

# Exactly equivalent, written without the pipe:
sdtm_lst <- lapply(
  list.files("~/R_info/sas2R_s/data/sdtm",
    pattern = "\\\\.sas7bdat$", full.names = TRUE),
  haven::read_sas
)

```

Step 3 — give the list meaningful names. Right now `sdtm_lst` is indexed by number; we want to call data sets by name (e.g. `sdtm_lst$ae`). We take the file names, strip the folder with `basename()`, and strip the extension with `gsub()`:

```

names(sdtm_lst) <- gsub(
  "\\\\.sas7bdat$", "",                # remove the extension
  basename(list.files("~/R_info/sas2R_s/data/sdtm",
    pattern = "\\\\.sas7bdat$", full.names = FALSE))
)

```

Step 4 — access data sets in the list. There are several ways to index:

```

sdtm_lst[[1]]           # first data set (by position)
sdtm_lst$ae             # the "ae" data set (by name)
sdtm_lst$ae[1, 1]       # row 1, column 1 of ae
sdtm_lst$ae[, "USUBJID"] # the whole USUBJID column of ae

```

Quick reminder on brackets (from the script's own note):

- `()` calls a function — `mean(x)`.
- `[]` subsets a vector, list, or data frame — `x[1]`, `df[, "AGE"]`.
- `[[ ]]` extracts a **single element** from a list — `sdtm_lst[["ae"]]`.
- `{ }` groups several expressions, e.g. a function body.

Inspecting and copying files

```

# Details (size, modified time, permissions) for every SAS file
file.info(list.files("~/R_info/sas2R_s/data/sdtm",
                    pattern = "\\sas7bdat$", full.names = TRUE))

# Back up all SAS files to another folder
file.copy(
  from = list.files("~/R_info/sas2R_s/data/sdtm",
                    pattern = "\\sas7bdat$", full.names = TRUE),
  to    = "~/R_info/sas2R_s/data/sdtm_backup"
)

file.exists("~/R_info/sas2R_s/data/sdtm_backup") # confirm the backup exists

```

Writing Your Own Functions

Once you repeat the same steps twice, wrap them in a function.

The basic syntax

```

# name <- function(arguments) { body }
test <- function(x) {
  y <- x + 1
  print(y)
}

test(2) # call the function with the argument 2
#> [1] 3

```

The four parts of a function:

1. **Name** — how you call it later (`test`).
2. **Arguments** — inputs in parentheses (`x`).
3. **Body** — the code in `{ }` that does the work.
4. **Return value** — what comes back. R returns the last evaluated expression; you can also be explicit with `return()`.

Documenting a function (`roxygen2`)

Special comments starting with `#'` are **roxygen2** documentation. When the function lives in a package, roxygen turns these into a proper help page (the thing you see when you type `?test`).

```

#' Test Function
#'
#' This function takes a numeric input and adds 1 to it.
#'

```

```

#' @param x A numeric value.
#' @return The input value incremented by 1.
#' @examples
#' test(2) # returns 3
#' test(5) # returns 6
#' @export
test <- function(x) {
  y <- x + 1
  print(y)
}

```

The tags mean: `@param` documents each argument, `@return` describes the output, `@examples` gives runnable examples, and `@export` makes the function visible to users of your package.

Three kinds of functions

- **Built-in** — ship with base R: `mean()`, `sum()`, `length()`.
- **User-defined** — ones you write with `function`, like `test()` above.
- **Package** — come from installed packages: `dplyr::filter()`, `ggplot2::ggplot()`.

Exploring Existing Packages and Functions

Before writing new code, learn what already exists. R is very introspectable — you can ask it about itself.

What is CRAN?

CRAN (the Comprehensive R Archive Network) is the central, official repository of R packages. When you run `install.packages("dplyr")`, R downloads it from CRAN. It stores the packages, their source code, and documentation.

Discovering a package's contents

```

library(help = "dplyr")    # opens a list of dplyr's functions and data sets
ls()                       # list objects in YOUR current environment
library(dplyr)             # load the package so its functions are available
ls("package:dplyr")        # list every function dplyr exposes
objects("package:dplyr")   # same as ls() - list the package's objects

```

Looking inside a specific function

```

args(dplyr::filter)        # what arguments does filter() accept?
body(dplyr::filter)         # show the actual source code of filter()

```

`args()` and `body()` are great learning tools: `args()` shows the “signature” (the inputs), and `body()` reveals how the function actually works.

Git Integration with RStudio

Git is version control — it saves snapshots (“commits”) of your project so you can track history and collaborate. **GitHub** is a website that hosts Git repositories online.

The everyday cycle (in RStudio’s Git pane)

1. **Initialize** a Git repository in your project directory.
2. **Create** a matching repository on GitHub and copy its URL.
3. In RStudio’s **Git** tab, click **Commit**, stage your changed files, and write a short message describing the change.
4. Click **Push** to upload your commits to GitHub.
5. Click **Pull** to bring down changes made by others.

One-time configuration

Tell Git who you are (this stamps every commit with your name):

```
library(usethis)
use_git_config(user.name = "My Username", user.email = "my_email@address.com")
```

You can also authenticate to GitHub. Modern GitHub uses either an SSH key (Tools → Global Options → Git/SVN → Create RSA Key) or a **Personal Access Token (PAT)**:

```
usethis::edit_git_config()      # open the git config file to inspect/edit
usethis::create_github_token()  # opens GitHub to mint a PAT
library(gitcreds)
gitcreds::gitcreds_set()        # paste the PAT to store it securely

# lower-level alternative to set config directly:
gert::git_config_set("user.email", "my_email@address.com")
```

Wiring a project to Git and GitHub from R

usethis automates the whole setup:

```
usethis::use_git()              # initialise the repo + make the first commit
usethis::use_r("my_new_script") # create R/my_new_script.R in a package
usethis::use_github()           # create the GitHub repo and connect it to your project
```

Building Your First Package

A **package** is the standard way to bundle functions, documentation, and tests so you (and others) can reuse them reliably.

Install the development toolkit

```
install.packages(c("devtools", "roxygen2", "usethis", "testthat"))
```

- **usethis** — automates project/package setup tasks.
- **roxygen2** — turns #' comments into help pages.
- **devtools** — the build/test/check/install workflow.
- **testthat** — the unit-testing framework.

Package-level documentation

Just as functions get roxygen comments, so does the whole package. A special block describes the package itself:

```
## My First R Package
#'
## This package contains functions for basic arithmetic operations.
#'
## @docType package
## @name mypackage
## @keywords package
## @examples
## test(2)
## test(5)
"_PACKAGE"
```

The development loop (recap)

```
usethis::create_package("~/projects/mypackage") # scaffold the package
usethis::use_r("test")      # create R/test.R and paste your function there
usethis::use_testthat()     # set up the tests/ folder
usethis::use_test("test")   # create a matching test file

devtools::document()        # build help pages + NAMESPACE from roxygen comments
devtools::load_all()        # load the package for interactive testing
devtools::test()            # run all tests
devtools::check()           # full R CMD check - the quality gate
devtools::install()         # install it into your R library
```

rlang: Tidy Evaluation for Beginners

The **rlang** package powers the “magic” in the tidyverse — the way **dplyr** lets you type a bare column name like `filter(df, AGE > 40)` without quotes. **rlang** gives you tools to **capture code**, **manipulate expressions**, and **write your own tidyverse-style functions**.

```
library(rlang)
library(dplyr)
```

Why it exists

When you write `filter(ads1_dev, AGE > 40)`, there is no object called `AGE` in your workspace — yet it works. `dplyr` **captures** the expression `AGE > 40` and evaluates it *inside* the data frame. This is **non-standard evaluation (NSE)**, and `rlang` is the toolkit that makes it possible.

Core rlang functions

```
# expr(): capture an expression WITHOUT running it
e <- expr(AGE + 1)
e
#> AGE + 1
class(e)           # a "call" - a stored piece of code
#> [1] "call"

# sym(): turn a string into a symbol (a bare name)
s <- sym("AGE")
s
#> AGE

# eval_tidy(): evaluate captured code inside a data context
eval_tidy(expr(AGE * 2), data = ads1_dev)
#> [1] 68 90 58 100

# as_label(): turn an expression/quosure back into a readable string
as_label(e)
#> [1] "AGE + 1"
```

Writing a function that takes a column name

The most useful beginner skill: a wrapper around `dplyr` that accepts a column name. The naive version fails because `dplyr` takes the literal word `col`:

```
avg_broken <- function(data, col) {
  summarise(data, avg = mean(col)) # looks for a column literally named "col"
}
avg_broken(ads1_dev, AGE)
#> Error in `summarise()`:
#> i In argument: `avg = mean(col)`.
#> Caused by error:
#> ! object 'AGE' not found
```

The fix is `{{ }}` (“curly-curly”), which says “use whatever the caller typed”:

```
avg <- function(data, col) {
  summarise(data, avg = mean({{ col }}))
}
avg(adsl_dev, AGE)    # works: AGE flows through correctly
#>      avg
#> 1 39.5
```

Under the hood, `{{ col }}` is shorthand for the classic two-step rlang idiom the script lists — `enquo()` to **capture**, then `!!` (“bang-bang”) to **unquote**:

```
avg_classic <- function(data, col) {
  col <- enquo(col)           # capture what the user typed
  summarise(data, avg = mean(!!col)) # inject it back into the dplyr call
}
avg_classic(adsl_dev, AGE)
#>      avg
#> 1 39.5
```

If the column name arrives as a **string** instead, use the `.data` pronoun:

```
avg_str <- function(data, col) {
  summarise(data, avg = mean(.data[[col]]))
}
avg_str(adsl_dev, "AGE")    # note the quotes
#>      avg
#> 1 39.5
```

The rlang “is_*” predicate family

The script lists many `is_*`() helpers. These are **type-checking predicates** — each returns a single TRUE/FALSE. They’re handy for input validation because they’re consistent and read clearly.

```
is_symbol(sym("AGE"))      # TRUE - it's a symbol
#> [1] TRUE
is_call(expr(mean(AGE)))   # TRUE - it's a function call
#> [1] TRUE
is_quosure(quo(AGE))       # TRUE - quo() makes a quosure
#> [1] TRUE
is_atomic(1:5)             # TRUE - a plain atomic vector
#> [1] TRUE
is_list(list(1, 2))        # TRUE
#> [1] TRUE
is_function(mean)          # TRUE
#> [1] TRUE
is_null(NULL)              # TRUE
#> [1] TRUE
is_empty(list())           # TRUE - length zero
#> [1] TRUE
is_named(c(a = 1, b = 2))  # TRUE - the vector has names
#> [1] TRUE
```

A quick tour of what the others check (from the script):

- `is_expression()` — an expression object.
- `is_formula()` — a formula like `y ~ x`.
- `is_vector()` / `is_data_frame()` / `is_tibble()` — container types.
- `is_missing()` — whether a function argument was left out.
- `is_finite()` / `is_infinite()` / `is_nan()` — numeric edge cases (`Inf`, `-Inf`, `NaN`).

These behave more predictably than some base R equivalents, which is why tidyverse code favours them.

Putting It All Together

A realistic mini-workflow that uses several ideas from this tutorial: compare two data sets, log the outcome with the messaging functions, and save the report.

```
library(diffrdf)

compare_and_log <- function(dev, qc, out_dir = "comparison_results") {
  # 1. make an output folder if needed
  if (!dir.exists(out_dir)) dir.create(out_dir)

  # 2. quick gate
  if (identical(dev, qc)) {
    message("Data sets are identical - nothing to report.")
    return(invisible(NULL))
  }

  # 3. detailed comparison
  result <- diffrdf(dev, qc)

  # 4. communicate + save
  warning("Differences found - review before proceeding.")
  saveRDS(result, file.path(out_dir, "diff_result.rds"))
  message("Comparison complete. Report saved to ", out_dir)

  invisible(result)
}

compare_and_log(ads1_dev, ads1_qc)
```

Summary

From the `package_dev.R` script you have now met:

- **Comparison** — `identical()`, `diffrdf()`, and `arsenal::comparedf()`.
- **Communication** — `print()`, `message()`, `warning()`, `stop()`, `warnings()`.
- **File handling** — `dir()`, `list.files()`, `dir.create()`, `unlink()`, `file.info()`, `file.copy()`, and reading many SAS files with `lapply()`.

- **Functions** — syntax, the four parts, and roxygen2 documentation.
- **Introspection** — `library(help=)`, `ls()`, `args()`, `body()`, and CRAN.
- **Git** — configuration and the commit/push/pull cycle via `usethis`.
- **Packages** — the `usethis` + `devtools` + `roxygen2` build loop.
- **rlang** — `expr()`, `sym()`, `enquo()`, `!!`, `{{ }}`, `.data[[ ]]`, `eval_tidy()`, and the `is_*`() predicate family.

The professional habit that ties it together: **write** → **document** → **test** → **commit** → **check** → **push**.

Further reading

- *R Packages* (2nd ed.) — <https://r-pkgs.org>
- *Advanced R* — <https://adv-r.hadley.nz>
- `diffdf` — <https://cran.r-project.org/package=diffdf>
- `arsenal` — <https://cran.r-project.org/package=arsenal>
- `rlang` / tidy evaluation — <https://rlang.r-lib.org> and <https://dplyr.tidyverse.org/articles/programming.html>
- Happy Git with R — <https://happygitwithr.com>